FPGA IMPLEMENTATION OF RISC-V PROCESSOR BASED SoC WITH MATRIX MULTIPLICATION ACCELERATOR

Technical Project Report

Project Type:	Self Directed FPGA and RTL Design Project
Processor:	RISC-V Based Processor
Hardware Description Language:	Verilog HDL
EDA Tool:	AMD Xilinx Vivado 2024.2
Target:	FPGA

2026

Abstract

This project presents the design and FPGA implementation of a modular RISC-V processor based System on Chip with a dedicated hardware accelerator for matrix multiplication.

The processor was developed using Verilog HDL and consists of independent RTL modules including the Program Counter, Instruction Decoder, Register File, Immediate Generator, Arithmetic Logic Unit, Memory subsystem, and finite state machine based control logic.

A dedicated matrix multiplication accelerator was developed to execute the computationally intensive matrix multiplication kernel directly in hardware. The accelerator contains BRAM based matrix storage, matrix address generation, multiplication, accumulation, index control, and result storage.

For matrices

$$A_{m\times n}$$

and

$$B_{n\times p},$$

the output matrix is

$$C_{m\times p}$$

where each output element is calculated using

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik} B_{kj}.$$

The complete design was developed incrementally, beginning with individual processor modules and progressing through memory integration, processor verification, accelerator development, subsystem verification, and FPGA implementation.

Contents

Abstract **1**

1 Introduction **7**

1.1 Project Motivation 7

1.2 Project Objectives 7

1.3 Development Methodology 8

2 System Architecture **9**

2.1 Overall Architecture 9

2.2 System Block Diagram 10

2.3 Major Components 10

3 RISC-V Processor Design **12**

3.1 Processor Datapath 12

3.2 Program Counter 12

3.3 Instruction Decoder 13

3.4 Register File 13

3.5 Immediate Generator 13

3.6 Arithmetic Logic Unit 14

3.7 Memory Operations 14

3.8 Processor Execution Flow 14

4 Memory Subsystem **16**

4.1 Memory Architecture 16

4.2 Instruction Memory 16
4.3 Data Memory 16
4.4 BRAM Based Matrix Storage 16

5 Matrix Multiplication Accelerator 17
5.1 Accelerator Motivation 17
5.2 Accelerator Architecture 18
5.3 Multiply Accumulate Operation 18
5.4 Accelerator Control Flow 18

6 Matrix Address Generation 21
6.1 Address Calculation 21
6.2 Address Generation Flow 22

7 Control Unit Design 23
7.1 Processor Control FSM 23
7.2 Accelerator Control FSM 24

8 Verification Methodology 25
8.1 Verification Strategy 25
8.2 Processor Verification 25
8.3 Accelerator Verification 26
8.4 Verification Flow 27

9 Matrix Multiplication Verification 28
9.1 Example Matrix Operation 28
9.2 Calculation of Output Elements 28

10 FPGA Implementation 30
10.1 Development Environment 30
10.2 FPGA Implementation Flow 30
10.3 Synthesis 31

10.4 Implementation 31

11 Design Characteristics **32**

11.1 Modular RTL Architecture 32

11.2 Dedicated Hardware Acceleration 32

11.3 BRAM Based Data Storage 32

11.4 Dedicated Address Generation 32

11.5 General Matrix Multiplication 33

12 Verification Summary **34**

13 Conclusion **35**

14 Project Summary **36**

A Key Equations **37**

A.1 Program Counter 37

A.2 Effective Address 37

A.3 Matrix A Address 37

A.4 Matrix B Address 37

A.5 Multiply Accumulate 37

A.6 Matrix Multiplication 38

List of Figures

1.1 Incremental development methodology 8

2.1 Overall architecture of the RISC-V based FPGA system 10

3.1 RISC-V processor execution flow 15

5.1 Matrix multiplication accelerator control flow 20

6.1 Matrix operand address generation 22

8.1 RTL verification and FPGA implementation flow 27

10.1 FPGA implementation flow 30

List of Tables

10.1 Development environment 30

12.1 Functional verification summary 34

14.1 Project summary 36

Chapter 1

Introduction

1.1 Project Motivation

General purpose processors are flexible, but computationally intensive operations can require a large number of instruction execution cycles.

Matrix multiplication is an important computational operation used in applications such as machine learning, computer vision, image processing, signal processing, and scientific computing.

A hardware accelerator can execute the repetitive multiplication and addition operations directly in dedicated FPGA logic.

The objective of this project is to combine a RISC-V processor architecture with a dedicated matrix multiplication accelerator.

1.2 Project Objectives

The main objectives of the project are:

1. Design a modular RISC-V processor using Verilog HDL.
2. Implement the Program Counter.
3. Implement instruction decoding.
4. Implement a register file.
5. Implement immediate generation.
6. Implement an Arithmetic Logic Unit.

7. Implement load and store operations.
8. Develop a BRAM based memory subsystem.
9. Develop a matrix multiplication accelerator.
10. Implement matrix address generation.
11. Implement multiplication and accumulation hardware.
12. Implement accelerator control using a finite state machine.
13. Verify the processor and accelerator at RTL level.
14. Prepare the complete design for FPGA implementation.

1.3 Development Methodology

The design was developed incrementally, with each major subsystem implemented and verified before integration with the next stage.

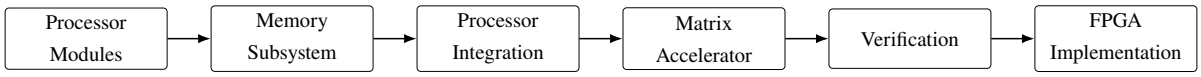


Figure 1.1: Incremental development methodology

Each stage was verified independently before being integrated with the subsequent subsystem. This incremental approach simplified debugging and allowed functional issues to be isolated at the module or subsystem level.

Chapter 2

System Architecture

2.1 Overall Architecture

The complete system contains two major computational sections.

The first section is the RISC-V processor subsystem responsible for general purpose instruction execution.

The second section is the matrix multiplication accelerator responsible for executing the matrix computation using dedicated hardware.

2.2 System Block Diagram

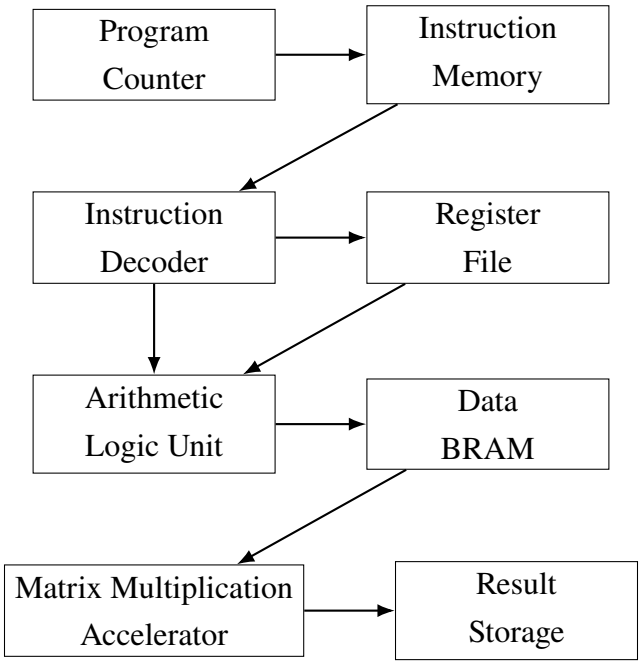


Figure 2.1: Overall architecture of the RISC-V based FPGA system

2.3 Major Components

The major components of the system are:

- Program Counter
- Instruction Memory
- Instruction Decoder
- Register File
- Immediate Generator
- Arithmetic Logic Unit
- Processor Control FSM
- Data Memory
- BRAM Interface
- Matrix Address Generator

- Multiplier
- Accumulator
- Accelerator Control FSM
- Result Storage

Chapter 3

RISC-V Processor Design

3.1 Processor Datapath

The RISC-V processor is organized as a modular datapath.

The major datapath elements are:

1. Program Counter
2. Instruction Memory
3. Instruction Decoder
4. Register File
5. Immediate Generator
6. Arithmetic Logic Unit
7. Data Memory
8. Writeback Logic
9. Control FSM

3.2 Program Counter

The Program Counter stores the address of the current instruction.

For sequential execution of 32 bit instructions, the next instruction address is

$$PC_{next} = PC + 4.$$

The Program Counter was implemented as an independent sequential RTL module.

3.3 Instruction Decoder

The instruction decoder extracts the fields required to determine the operation being executed.

The important fields include:

- Opcode
- Destination register
- Source register 1
- Source register 2
- funct3
- funct7
- Immediate fields

The decoder provides the information required by the datapath and control logic.

3.4 Register File

The processor uses 32 general purpose registers.

The register file provides two source operands and supports destination register writeback.

Register zero follows the RISC-V architectural requirement.

3.5 Immediate Generator

The immediate generator extracts immediate fields from instructions and performs the required sign extension.

Different RISC-V instruction formats use different immediate encodings.

3.6 Arithmetic Logic Unit

The Arithmetic Logic Unit performs arithmetic and logical operations.

The implemented operations include:

- Addition
- Subtraction
- AND
- OR
- XOR
- Comparison

3.7 Memory Operations

Load and store operations use an effective address.

The effective address is calculated as

$$EffectiveAddress = BaseAddress + Offset.$$

For a load instruction, the memory data is transferred into the destination register.

For a store instruction, register data is written into memory.

3.8 Processor Execution Flow

The processor execution sequence is illustrated in Figure 3.1.

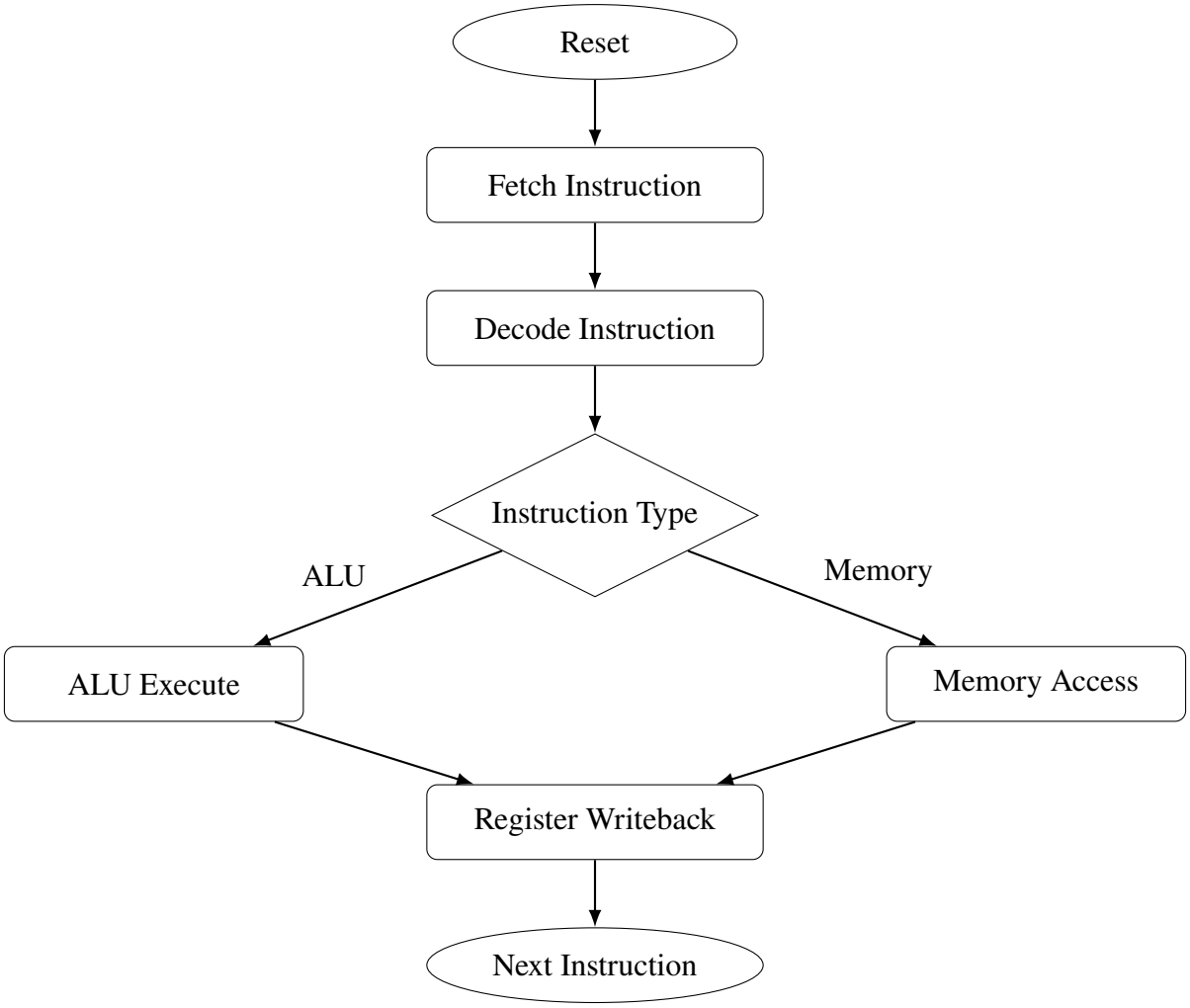


Figure 3.1: RISC-V processor execution flow

Chapter 4

Memory Subsystem

4.1 Memory Architecture

The memory subsystem provides storage for instructions, processor data, and matrix operands.

FPGA Block RAM is used as local on chip storage.

4.2 Instruction Memory

Instruction memory stores the program instructions executed by the processor.

The Program Counter supplies the instruction address.

4.3 Data Memory

Data memory supports processor load and store operations.

The memory subsystem also provides access to matrix data required by the accelerator.

4.4 BRAM Based Matrix Storage

Matrix operands are loaded into BRAM before accelerator execution.

The accelerator generates addresses using the matrix dimensions and current matrix indices.

This allows the matrix elements to be accessed during each multiply accumulate operation.

Chapter 5

Matrix Multiplication Accelerator

5.1 Accelerator Motivation

Matrix multiplication contains a large number of repetitive multiplication and addition operations.

For

$$A_{m \times n}$$

and

$$B_{n \times p},$$

the result is

$$C_{m \times p}.$$

Each output element is calculated as

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik} B_{kj}.$$

A dedicated accelerator executes these operations directly in hardware.

5.2 Accelerator Architecture

The accelerator consists of:

- Matrix address generation logic
- BRAM interface
- Multiplication unit
- Accumulator
- Row counter
- Column counter
- Inner product counter
- Accelerator control FSM
- Output storage

5.3 Multiply Accumulate Operation

The central arithmetic operation of the accelerator is

$$ACC_{next} = ACC + A_{ik}B_{kj}.$$

For each value of k, the corresponding elements of matrices A and B are multiplied.

The product is added to the accumulator.

After the complete inner product is calculated,

$$C_{ij} = ACC.$$

The accumulator is then cleared before calculating the next output element.

5.4 Accelerator Control Flow

The accelerator uses three matrix indices:

$$i, \qquad j, \qquad k.$$

The index i selects the output row, j selects the output column, and k traverses the inner product.

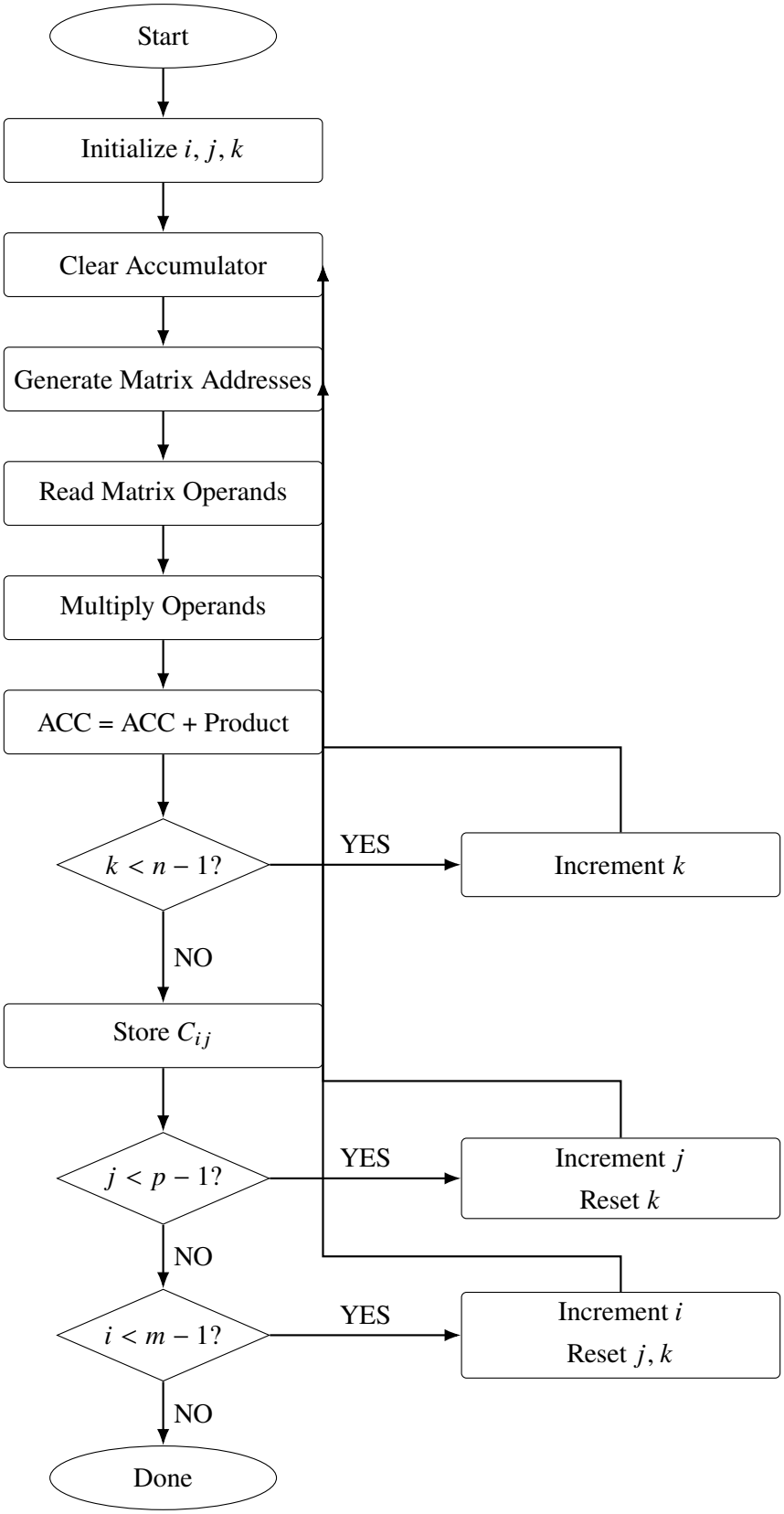


Figure 5.1: Matrix multiplication accelerator control flow

Chapter 6

Matrix Address Generation

6.1 Address Calculation

For row major storage, the address of matrix A element A_{ik} is

$$Address_A = Base_A + (i \times n + k) \times W.$$

The address of matrix B element B_{kj} is

$$Address_B = Base_B + (k \times p + j) \times W.$$

where W represents the number of bytes occupied by one matrix element.

6.2 Address Generation Flow

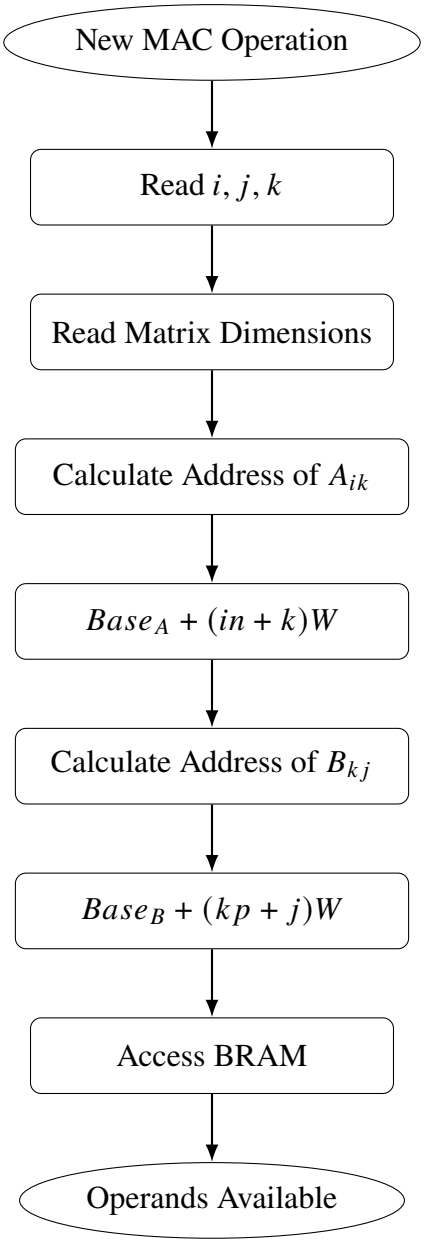


Figure 6.1: Matrix operand address generation

Chapter 7

Control Unit Design

7.1 Processor Control FSM

The processor control unit uses a finite state machine to coordinate instruction execution.

The main execution phases are:

1. Instruction Fetch
2. Instruction Decode
3. Execute
4. Memory Access
5. Writeback

The control unit generates signals for:

- Memory read
- Memory write
- Register write
- ALU control
- Memory to register selection
- Execution type

7.2 Accelerator Control FSM

The accelerator has a dedicated control sequence.

The FSM coordinates:

1. Initialization
2. Address generation
3. Memory access
4. Multiplication
5. Accumulation
6. Index update
7. Result storage
8. Completion

Separating the accelerator FSM from the processor FSM keeps the processor datapath modular.

Chapter 8

Verification Methodology

8.1 Verification Strategy

Verification was performed progressively at RTL level.

Individual modules were tested before subsystem integration.

The verification sequence was:

Module Verification → Subsystem Verification → Processor Verification → Accelerator Verification → Syst

8.2 Processor Verification

The processor verification included:

- Program Counter reset
- Program Counter update
- Instruction decoding
- Register file read operation
- Register file write operation
- Immediate generation
- ALU operations
- Load operation

- Store operation
- Control FSM operation

8.3 Accelerator Verification

The accelerator verification included:

- Reset operation
- BRAM initialization
- Matrix operand loading
- Matrix address generation
- Operand retrieval
- Multiplication
- Accumulator update
- Matrix index progression
- Output storage
- Completion behavior

8.4 Verification Flow

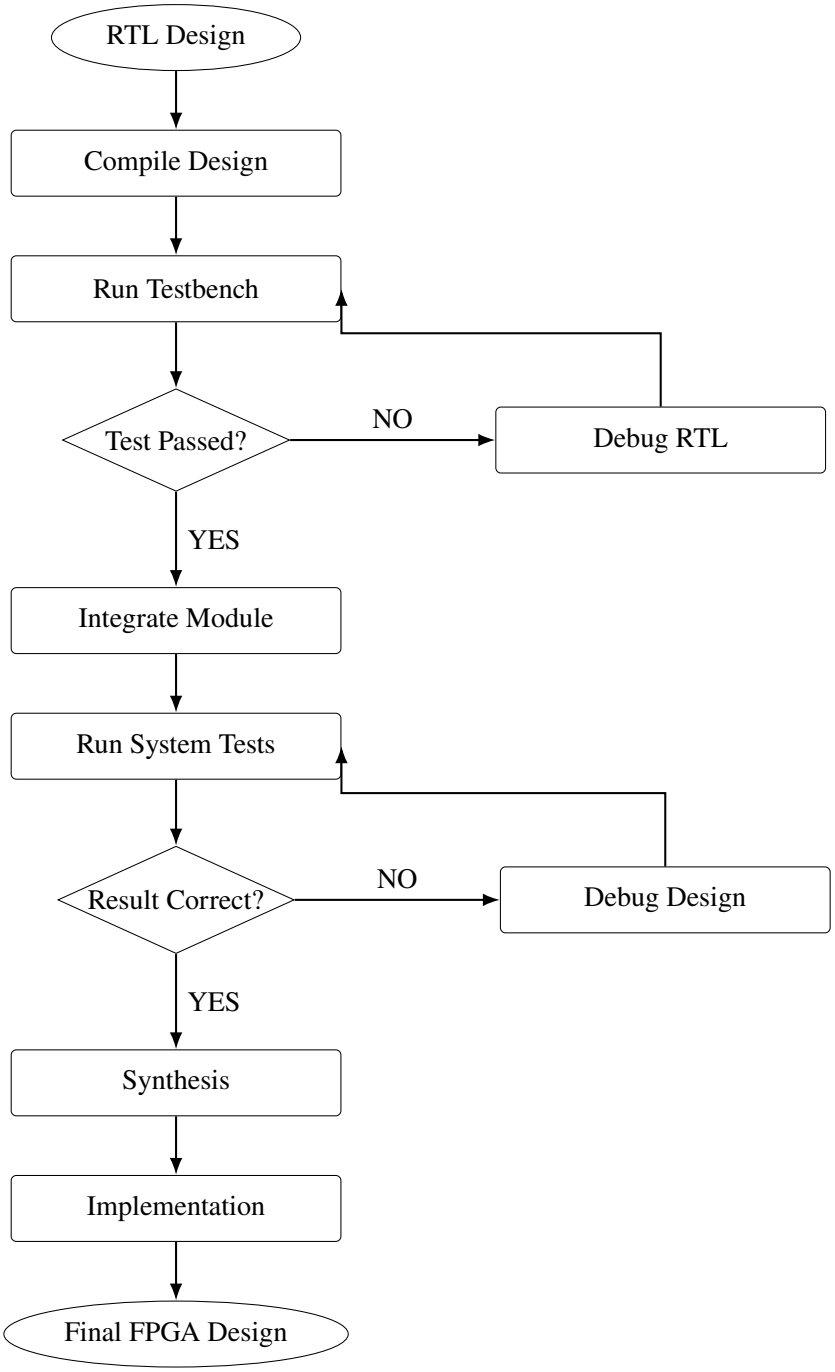


Figure 8.1: RTL verification and FPGA implementation flow

Chapter 9

Matrix Multiplication Verification

9.1 Example Matrix Operation

Consider the matrices

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

and

$$B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

The expected result is

$$C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}.$$

9.2 Calculation of Output Elements

The first element is

$$C_{00} = (1)(5) + (2)(7) = 19.$$

The second element is

$$C_{01} = (1)(6) + (2)(8) = 22.$$

The third element is

$$C_{10} = (3)(5) + (4)(7) = 43.$$

The fourth element is

$$C_{11} = (3)(6) + (4)(8) = 50.$$

Therefore,

$$\boxed{C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}}$$

The accelerator result is compared with the expected mathematical result during RTL verification.

Chapter 10

FPGA Implementation

10.1 Development Environment

The design was developed using Verilog HDL and AMD Xilinx Vivado 2024.2.

Table 10.1: Development environment

Parameter	Value
Hardware Description Language	Verilog HDL
EDA Tool	AMD Xilinx Vivado 2024.2
RTL Simulation	Vivado Simulator
Synthesis	Vivado Synthesis
Implementation	Vivado Implementation
Target Platform	FPGA

10.2 FPGA Implementation Flow

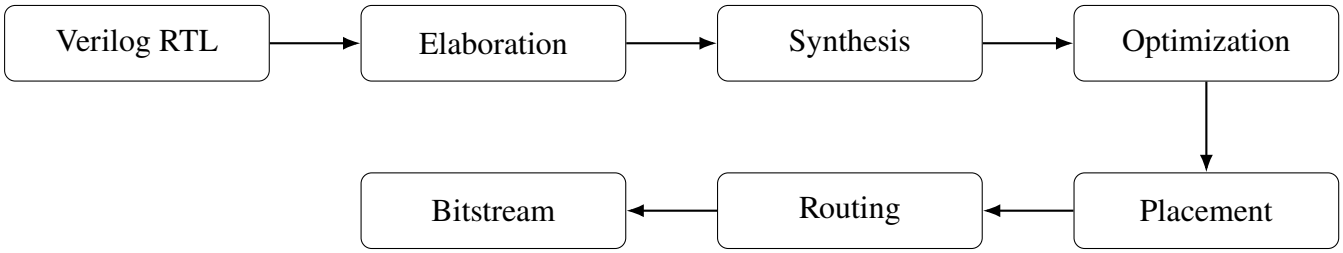


Figure 10.1: FPGA implementation flow

10.3 Synthesis

During synthesis, the Verilog RTL is converted into a technology specific FPGA implementation.

The design may utilize FPGA resources including:

- Look Up Tables
- Flip Flops
- Block RAM
- DSP resources
- Input and Output resources

10.4 Implementation

The synthesized design is placed and routed on the target FPGA.

Timing analysis and resource utilization analysis are performed after implementation.

Chapter 11

Design Characteristics

11.1 Modular RTL Architecture

The processor and accelerator are divided into independent RTL modules.

This improves design readability, debugging, verification, and future extensibility.

11.2 Dedicated Hardware Acceleration

The matrix multiplication kernel is executed using dedicated hardware.

The processor therefore does not need to perform every individual multiply and accumulate operation through general purpose instructions.

11.3 BRAM Based Data Storage

BRAM is used for local storage of matrix operands.

This provides predictable on chip data access for the accelerator.

11.4 Dedicated Address Generation

Matrix addresses are generated from the matrix dimensions and the current matrix indices.

For matrix A,

$$Address_A = Base_A + (i \times n + k) \times W.$$

For matrix B,

$$Address_B = Base_B + (k \times p + j) \times W.$$

11.5 General Matrix Multiplication

The accelerator follows the general matrix operation

$$A_{m \times n} \times B_{n \times p} = C_{m \times p}.$$

The architecture is therefore applicable to matrices with different dimensions subject to available memory and FPGA resources.

Chapter 12

Verification Summary

Table 12.1: Functional verification summary

Verification Item	Status
Program Counter reset	PASS
Program Counter update	PASS
Instruction decoding	PASS
Register file operation	PASS
Immediate generation	PASS
ALU arithmetic operations	PASS
ALU logical operations	PASS
Load operation	PASS
Store operation	PASS
BRAM initialization	PASS
Matrix address generation	PASS
Multiplier operation	PASS
Accumulator operation	PASS
Accelerator FSM	PASS
Matrix multiplication	PASS

Chapter 13

Conclusion

This project developed a modular RISC-V processor based FPGA system with a dedicated matrix multiplication accelerator using Verilog HDL.

The processor contains the Program Counter, Instruction Decoder, Register File, Immediate Generator, Arithmetic Logic Unit, memory subsystem, and finite state machine based control logic.

The matrix multiplication accelerator integrates BRAM based matrix storage, address generation, multiplication, accumulation, index control, and output storage.

The design was developed incrementally, starting from individual RTL modules and progressing through memory integration, processor verification, accelerator development, subsystem verification, and FPGA implementation.

The final architecture demonstrates how a general purpose RISC-V processor can be combined with a dedicated hardware accelerator to execute computationally intensive matrix operations.

Chapter 14

Project Summary

Table 14.1: Project summary

Category	Implementation
Processor Architecture	RISC-V based processor
HDL	Verilog HDL
Program Counter	Dedicated RTL module
Instruction Processing	Fetch, decode and execution
Register File	32 general purpose registers
ALU	Arithmetic and logical operations
Memory	BRAM based memory subsystem
Control	Finite state machine based control
Accelerator	Matrix multiplication accelerator
Arithmetic Kernel	Multiply accumulate
Address Generation	Index based matrix addressing
Verification	RTL testbench based verification
EDA Tool	AMD Xilinx Vivado 2024.2
Target	FPGA

Appendix A

Key Equations

A.1 Program Counter

$$PC_{next} = PC + 4$$

A.2 Effective Address

$$EffectiveAddress = BaseAddress + Offset$$

A.3 Matrix A Address

$$Address_A = Base_A + (i \times n + k) \times W$$

A.4 Matrix B Address

$$Address_B = Base_B + (k \times p + j) \times W$$

A.5 Multiply Accumulate

$$ACC_{next} = ACC + A_{ik}B_{kj}$$

A.6 Matrix Multiplication

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik} B_{kj}$$